

Deep and Reinforcement Learning

Daniel Carneiro
Faculty of Engineering
University of Porto
Porto, Portugal
up202108832@edu.fe.up.pt

David Pinto
Faculty of Engineering
University of Porto
Porto, Portugal
up201704300@edu.fc.up.pt

José Santos
Faculty of Engineering
University of Porto
Porto, Portugal
up202108729@edu.fe.up.pt

Abstract—In recent years, the emergence of deep learning has led to significant advances in image synthesis, allowing the creation of highly realistic but artificial images commonly referred to as ‘deep fakes’. This work aims to explore the dual facets of the deep learning model in the context of image generation by developing discriminative and generative models. In parallel, generative modeling is employed to produce realistic synthetic images. To achieve this, we created several different models based on convolutional neural networks, generating attention networks and variational autoencoders adapted to each objective. In addition, we evaluated the performance of our models in terms of Area Under Curve (AUC), Equal Error Rate (ERR), and Accuracy (Acc). At the end of the classifier task, we conclude that CNNs are highly effective for image classification.

I. INTRODUCTION

In recent years, the emergence of deep learning has led to significant advances in image synthesis, allowing the creation of highly realistic but artificial images commonly referred to as ‘deep fakes’. Although these technologies have opened new frontiers in media generation and entertainment, they have also raised serious concerns about misinformation, digital forgery, and identity misuse.

This work aims to explore the dual facets of deep learning in the context of image generation by developing discriminative and generative models. The discriminative models are designed to classify images as either “real” or “fake”. In parallel, generative models are employed to produce realistic synthetic images. The dataset ‘DeepFakeFace’ (DFF) [7] published by OpenRL will be the main source of data where we were provided a hundred and twenty thousand images where thirty thousand are real images and the other ninety thousand are fake images equally split between three generative models (Stable Diffusion v1.5, Stable Diffusion InPainting and InsightFace).

By addressing both creation and detection, this report presents a comprehensive investigation into the capabilities and limitations of deep learning models in generating and classifying images.

II. CLASSIFIER

The classifier modeling component focuses on training models to classify images as real or fake by learning from both real and synthetic images using the 120,000 images available in the dataset. To achieve some results, three different models were implemented: a simple Convolutional Neural Network

(CNN), another CNN where improvements were made and finally a pretrained model using a Residual Network (ResNet-18) [12]. Each model was trained and evaluated in terms of Area Under Curve (AUC), Equal Error Rate (ERR) and Accuracy (Acc).

A. Methodology

The classifier must be able to analyze the image as a whole and identify possible patterns that may indicate whether the image is fake or not. To support this, we created six distinct datasets, each with unique preprocessing steps designed to emphasize different visual features and assess their impact on model learning. These included:

- **Standard dataset** with only resizing to (224,224);
- **Grayscale dataset** with resizing to (224,224) and 3-channel grayscale image;
- **Contrast-enhanced dataset** with resizing to (224,224) and contrast level 20;
- **Saturation-enhanced dataset** with resizing to (224,224) and saturation level 10;
- **Gaussian blur dataset** with resizing to (224,224) and using a kernel size of (31, 31);
- **Pixelated dataset** produced by downscaling and then resizing to (224, 224).

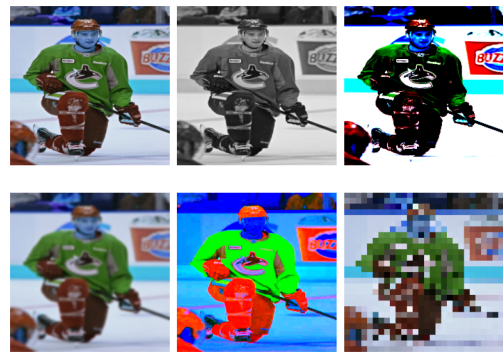


Fig. 1: Image preprocessing

Additionally, we developed two CNNs, and used ResNet-18 to compare the performance of our models.

III. CONVOLUTIONAL NEURAL NETWORK (CNN)

A Convolutional Neural Network (CNN) is a type of deep learning model particularly well-suited for image analysis

tasks. CNNs are designed to automatically and adaptively learn spatial hierarchies of features from input images through the use of convolutional layers, pooling layers, and fully connected layers. These models are effective at capturing patterns such as edges, textures, and complex structures, making them ideal for detecting subtle differences between real and fake images. Following this principle, we implemented and evaluated three different CNN-based models to classify images as real or fake:

A. Simple CNN model

The Simple CNN is a basic convolutional neural network designed to serve as a baseline for image classification tasks. Its architecture consists of three convolutional layers, followed by batch normalization, ReLU activation, and max pooling. After feature extraction through convolutional layers, a Global Average Pooling layer reduces the feature maps to a single value per channel, and a fully connected layer produces the final classification output.

• Architecture Overview:

- **Convolutional Layers:** The model starts with three convolutional layers:
 - * The first layer has 32 filters, followed by batch normalization and ReLU activation.
 - * The second layer has 64 filters, again followed by batch normalization and ReLU.
 - * The third layer has 128 filters, with batch normalization and ReLU activation.
 All convolutional layers use a 3x3 kernel with padding, ensuring that the spatial dimensions are preserved after each convolution.
- **Max Pooling:** After each convolutional block, a max pooling layer is applied with a 2x2 kernel, which reduces the spatial size by half, effectively downsampling the feature maps.
- **Global Average Pooling:** The output of the convolutional layers is passed through a Global Average Pooling layer, which reduces each feature map to a single value (1x1 per channel), providing a compact representation of the learned features.
- **Fully Connected Layer:** A fully connected layer (fc) is applied to the pooled features, transforming the output from 128 features to num_classes (2 in this case for real vs fake classification).
- **Softmax Activation:** The output is passed through a softmax activation function to obtain class probabilities, with the model's final prediction corresponding to the class with the highest probability.

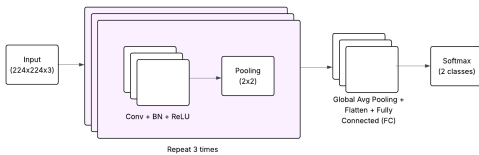


Fig. 2: Simple CNN flow Diagram

B. Dropout CNN model

The Dropout CNN is a modified version of the Simple CNN, that was created by experimenting on the Simple CNN by adding iteratively 2 more layers that increase the complexity of our model. We also incorporated dropout to the layers so the model can reduce the risk of overfitting and improve the model's ability to generalize to unseen data. By randomly deactivating a fraction of neurons during training, dropout helps prevent the model from becoming overly reliant on any specific features, thus encouraging it to learn more robust and diverse representations.

• Architecture Overview:

- **Convolutional Layers:** Similar to the Simple CNN, the Dropout CNN starts with five convolutional layers:
 - * The first layer has 32 filters, followed by batch normalization and ReLU activation.
 - * The second layer has 64 filters, again followed by batch normalization and ReLU.
 - * The third layer has 128 filters, with batch normalization and ReLU activation.
 - * The fourth layer has 256 filters, followed by batch normalization and ReLU.
 - * The fifth layer has 512 filters, with batch normalization and ReLU activation.

All convolutional layers use a 3x3 kernel with padding to preserve the spatial dimensions of the input.

- **Max Pooling:** After each convolutional block, a max pooling layer is applied with a 2x2 kernel and a stride of 2, which reduces the spatial size by half and downscales the feature maps.
- **Dropout:** After each convolutional layer (except the last), a dropout layer is applied with a dropout rate of 0.1. This ensures that 10% of the neurons are randomly deactivated during training, helping the model avoid overfitting by preventing it from relying too heavily on any individual feature.
- **Global Average Pooling:** The output of the convolutional layers is passed through a Global Average Pooling layer, which reduces each feature map to a single value, effectively summarizing the learned features into a compact form.
- **Fully Connected Layer:** A fully connected layer is applied to the pooled features, transforming the output from 512 features to num_classes (2 for real vs fake classification).
- **Softmax Activation:** The final output is passed through a softmax activation function to compute the probabilities for each class, with the model's prediction corresponding to the class with the highest probability.

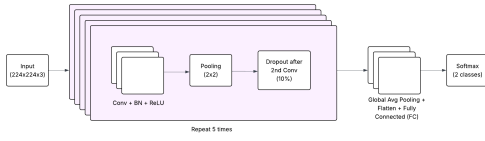


Fig. 3: Dropout CNN flow Diagram

C. Resnet-18

ResNet-18 is a deep convolutional neural network based on the Residual Network (ResNet) architecture. ResNet-18 is specifically designed to solve the problem of vanishing gradients in very deep networks by introducing residual connections (or skip connections) that allow gradients to flow directly through layers, bypassing some intermediate layers. This facilitates the training of deeper networks by making it easier to propagate information across many layers. The ResNet-18 model used in this study is pretrained, allowing it to benefit from prior learning on a large dataset, which can be fine-tuned for our specific task of detecting real vs fake images.

• Architecture Overview:

- **Residual Connections:** The hallmark of the ResNet architecture is its use of skip connections between layers. These residual connections allow the input to bypass certain layers and be added directly to the output, which helps to prevent the degradation of performance as the network depth increases. In ResNet-18, these connections are implemented in the form of bottleneck blocks that consist of a series of convolutions and add the input directly to the output.
- **Convolutional Layers:** ResNet-18 consists of 18 layers in total, including convolutional layers, batch normalization, and activation functions. Unlike the simple CNN, ResNet-18 includes a combination of:
 - * **Initial Convolution:** The first layer is a convolutional layer that processes the input image with a 7x7 kernel, followed by batch normalization and a ReLU activation function.
 - * **Residual Blocks:** The subsequent layers are organized into blocks, each consisting of two 3x3 convolutions with residual connections, allowing information to flow more easily through the network.
 - * **Downsampling:** Downsampling is performed in certain residual blocks to reduce the spatial dimensions of the feature maps.
- **Batch Normalization:** Like the previous models, ResNet-18 uses batch normalization after each convolution to stabilize training by normalizing the activations and helping to mitigate internal covariate shift.
- **Fully Connected Layer:** After feature extraction through the residual blocks, the feature maps are passed through a global average pooling layer to

reduce the spatial dimensions to a single value per channel. The pooled output is then flattened and passed through a fully connected layer to map the 512 features to the final class prediction.

- **Softmax Activation:** The final output layer uses the softmax activation function to convert the raw output values into probabilities for each class, with the highest probability determining the model's prediction.

IV. CLASSIFIER RESULTS

As mentioned previously, we evaluated the models with the metrics Area Under Curve (AUC), Equal Error Rate (ERR) and Accuracy (Acc). However, before we start analyzing them, we think we should mention the evolution of the loss reward that we decided to use (`nn.CrossEntropyLoss()`). As seen in figure 4, all models tend to decrease the loss as the epochs occur. In this way, we can see that the models are managing to learn something over time.

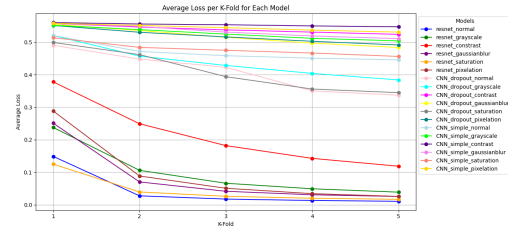


Fig. 4: Average Loss per K-Fold for Each Model

After the execution of each model, it was necessary to evaluate the models. As such, Accuracy, AUC and EER were used. It is observable that the accuracy and AUC increase over time while EER decreases, this happens especially in the resnet models, particularly due to the pretrained weights.

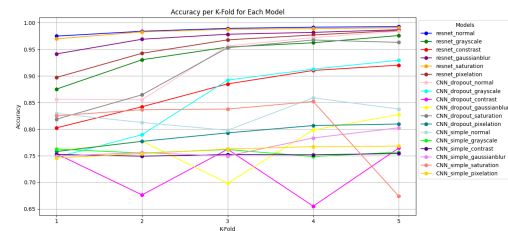


Fig. 5: Accuracy per K-Fold for Each Model

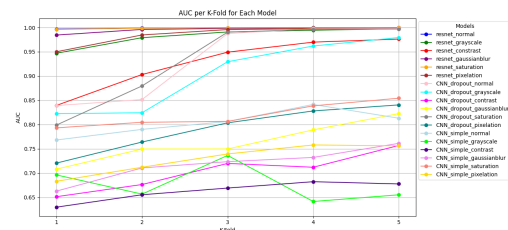


Fig. 6: AUC per K-Fold for Each Model

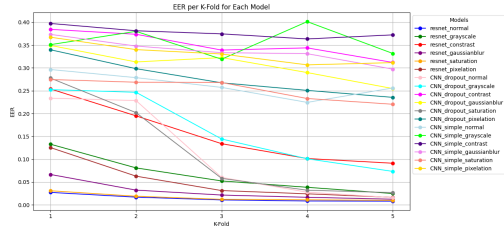


Fig. 7: EER per K-Fold for Each Model

After verifying the excellent results of RESNET, we decided to create our own neural network to see if our models could achieve equally good results. Although our first model (CNN_simple) performed poorly, after some research we discovered that the utilization of dropout to turn off some connections would make the model less prone to overfitting. Also, the addition of two more layers can make the model achieve more complex patterns. The result were clearly much better, with a few epochs the model managed to achieve an impressive 92.5% accuracy and 93% AUC, which means that the model was able to recognize the differences between real and fake images well. Having a 10% improvement in accuracy and 13% in AUC compared to CNN_simple_normal (simple model that uses the untreated dataset). Finally, after comparing these 3 models, we decided to apply some pre-processing to the images to check if this would impact the model's learning. From the 6 datasets we used, normal and saturated datasets achieved the better results, as you can see in the table 1.

TABLE I: Models Final Results

Model	Accuracy	AUC	EER
resnet_normal	0.9866	0.9990	0.0143
resnet_grayscale	0.9397	0.9818	0.0659
resnet_contrast	0.8721	0.9275	0.1551
resnet_gaussianblur	0.9716	0.9954	0.0299
resnet_saturation	0.9840	0.9987	0.0166
resnet_pixelation	0.9541	0.9854	0.0521
CNN_dropout_normal	0.9249	0.9347	0.1132
CNN_dropout_grayscale	0.8543	0.9035	0.1635
CNN_dropout_contrast	0.7224	0.7032	0.3507
CNN_dropout_gaussianblur	0.7722	0.7637	0.3059
CNN_dropout_saturation	0.9135	0.9324	0.1193
CNN_dropout_pixelation	0.7889	0.7913	0.2784
CNN_simple_normal	0.8274	0.8038	0.2626
CNN_simple_grayscale	0.7569	0.6771	0.3567
CNN_simple_contrast	0.7520	0.6627	0.3779
CNN_simple_gaussianblur	0.7682	0.7180	0.3367
CNN_simple_saturation	0.8051	0.8194	0.2529
CNN_simple_pixelation	0.7595	0.7296	0.3313

Seeing the results as a whole and believing that our last model could reach similar results to resnet we decided to train it one last time with cross validation of 5-fold with 20 epochs each.

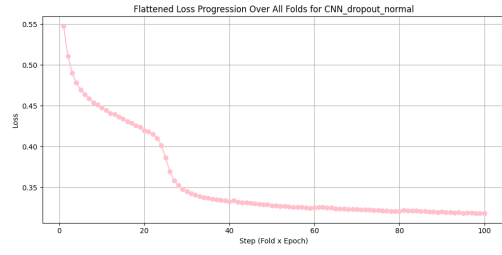


Fig. 8: cnn_dropout_normal loss results

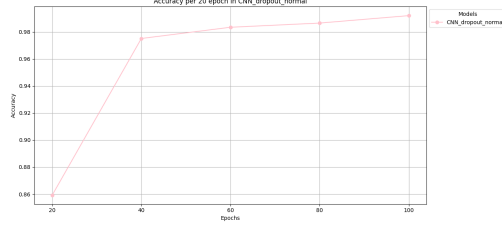


Fig. 9: cnn_dropout_normal accuracy results

As we expected, our model came very close to ResNet-18 after just a few epochs, reaching an accuracy of 99.2%, only 0.08% lower than ResNet. Figures 8 and 9 clearly show the model's learning progression and the corresponding decrease in loss.

V. GENERATOR

The generative modeling component of our project focuses on training models to produce synthetic facial images by learning from real data. Using the 30,000 real images from the DFF dataset [7], the goal is to replicate the visual and statistical properties of authentic faces. To explore different generative strategies, three models were implemented: two Deep Convolutional GANs (DCGAN) [13] and a Variational Autoencoder (VAE) [10]. Each model was trained and iteratively refined to improve generation quality, with outputs evaluated in terms of visual fidelity and consistency with the synthetic styles present in the DFF dataset [7].

A. Methodology

Along with the development and testing of the generative part of our project, we followed an organized step-by-step approach. This process was divided into several key phases:

- **Simple DCGAN [13] with Reduced Images:** This baseline model together with the resizing of the dataset images served as our reference point and helped to understand the fundamental behavior of the DCGAN.
- **Advanced Preprocessing with Face Cropping:** To improve the preprocessing part of our work, we used a Multi-Task Cascaded Convolutional Networks (MTCNN) [8] to automatically crop faces from the input images. This approach made sure that the generator and the discriminator were solely focused on the facial features of each image, reducing the background bloat and enhancing the quality of the generated images.

- **Hyperparameter and Model Optimization:** Following the new preprocessing, we performed some hyperparameter tuning to try to refine our model. Key adjustments included:
 - **Learning Rate Reduction:** Lowered the learning to achieve a more stable convergence during training.
 - **Model Depth Augmentation:** Added extra layers and an attention channel to improve the model’s capacity to learn complex features and generate quality images.
- **Variational Autoencoder (VAE) [10] Implementation:** After the successful implementation of the GANs, we widened our research and attention to the GAN’s. Using the generated cropped images, VAEs were explored as an alternative, allowing us to compare the probabilistic modeling capabilities of VAEs with the adversarial training of GANs.

VI. SIMPLE DCGAN [13] WITH REDUCED IMAGES

A. Image Preprocessing

The image preprocessing is fundamental since it prepares all the data for the generator modeling and evaluation iterations. The following preprocessing pipeline was implemented:

- **Resizing:** All images were resized to a 128×128 resolution, ensuring that the Generator receives inputs with uniform dimensions and reduces the computational complexity.
- **Tensor Conversion:** All images were converted into ‘PyTorch’ tensors, so that the image has suitable format.
- **Normalization:** This normalization shifts the pixel value range to approximately $[-1, 1]$ by subtracting a mean of 0.5 and dividing by a standard deviation of 0.5 for each channel. Such normalization is advantageous, as it contributes to stabilizing the learning process.

B. Generator Architecture

The generator transforms a latent vector of size 100 into a 128×128 RGB image. The architecture comprises two main stages: a latent transformation and a sequence of upsampling blocks.

1) **Latent Transformation:** The generator begins with a latent vector $z \in \mathbb{R}^{100}$. This vector is passed through a linear layer that maps it to a vector of dimension $128 \times 8 \times 8$. A ReLU activation function is applied, and the resulting vector is reshaped into a tensor of dimensions $(128, 8, 8)$.

2) **Upsampling Blocks:** The reshaped tensor is progressively upsampled to reach the final output resolution of 128×128 . The process involves the following blocks:

a) Block 1:

- **Upsampling:** Upsamples the 8×8 tensor to 16×16 .
- **Convolution:** A ConvTranspose2d layer with a 3×3 kernel and padding of 1 is applied, maintaining 128 channels.
- **Normalization & Activation:** Batch normalization and a ReLU activation follow.

b) Block 2:

- **Upsampling:** Upsamples the 16×16 tensor to 32×32 .
- **Convolution:** A ConvTranspose2d layer with a 3×3 kernel and padding of 1 is employed to maintain 128 channels.
- **Normalization & Activation:** Batch normalization and a ReLU activation are applied.

c) Block 3:

- **Upsampling:** Upsamples the tensor from 32×32 to 64×64 .
- **Convolution:** A ConvTranspose2d layer with a 3×3 kernel and padding of 1 reduces the number of channels from 128 to 64.
- **Normalization & Activation:** Batch normalization and a ReLU activation follow.

d) Block 4:

- **Upsampling:** Upsamples the tensor from 64×64 to 128×128 .
- **Convolution:** A ConvTranspose2d layer with a 3×3 kernel and padding of 1 adjusts the channels from 64 to 3, corresponding to the RGB output.
- **Activation:** A Tanh activation function is applied to ensure that the output pixel values are in the range $[-1, 1]$.

C. Discriminator Architecture

The discriminator processes an input RGB image coming from the Generator through a series of convolutional blocks with increasing feature depth, followed by a final fully connected layer that outputs a probability.

a) Block 1:

- **Convolution:** Applies a Conv2d layer mapping from 3 to 32 channels using a 3×3 kernel, stride of 2, and padding of 1.
- **Activation:** A LeakyReLU activation with a negative slope of 0.2 is applied.
- **Regularization:** Dropout is used with a probability of 0.25.

b) Block 2:

- **Convolution:** Applies a Conv2d layer mapping from 32 to 64 channels with a 3×3 kernel, stride of 2, and padding of 1.
- **Normalization:** Batch Normalization is applied with a momentum of 0.8.
- **Activation:** A LeakyReLU activation with a negative slope of 0.25 is applied.
- **Regularization:** Dropout is used with a probability of 0.25.

c) Block 3:

- **Convolution:** Applies a Conv2d layer mapping from 64 to 128 channels using a 3×3 kernel, stride of 2, and padding of 1.
- **Normalization:** Batch Normalization is applied with a momentum of 0.8.
- **Activation:** A LeakyReLU activation with a negative slope of 0.2 is applied.

$$-\frac{1}{n} \sum_{i=1}^n y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

Fig. 10: Standard Binary Cross-Entropy

- **Regularization:** Dropout is used with a probability of 0.25.

d) **Block 4:**

- **Convolution:** Applies a Conv2d layer mapping from 128 to 256 channels using a 3×3 kernel, stride of 2, and padding of 1.
- **Normalization:** Batch Normalization is applied with a momentum of 0.8.
- **Activation:** A LeakyReLU activation with a negative slope of 0.25 is applied.
- **Regularization:** Dropout is used with a probability of 0.25.

e) **Block 5:**

- **Convolution:** Applies a Conv2d layer mapping from 256 to 512 channels using a 3×3 kernel, stride of 2, and padding of 1.
- **Normalization:** Batch Normalization is applied with a momentum of 0.8.
- **Activation:** A LeakyReLU activation with a negative slope of 0.2 is applied.
- **Regularization:** Dropout is used with a probability of 0.25.

f) **Final Classification Layer:**

- **Flattening:** The output feature map is flattened.
- **Fully Connected Layer:** A Linear layer maps the flattened features (of size $512 \times 4 \times 4$) to a single scalar.
- **Activation:** A Sigmoid activation function is applied to produce a probability output.

D. Loss Function

The primary Loss Function used was the binary cross-entropy (BCE) [14]. The Discriminator outputs values close to 1 for real images and 0 for the fake ones generated by the Generator. The Generator will try to always keep values closer to one, so this adversarial setup ensures that both structures improve iteratively.

E. Optimiser and Hyperparameters

The training uses the Adam optimizer for both the Generator and Discriminator with the following hyperparameters:

- Latent Dimension: 100
- Learning Rate: 0.0001
- Beta1: 0.5
- Beta2: 0.999
- Number of Epochs: 100

F. Training Outputs

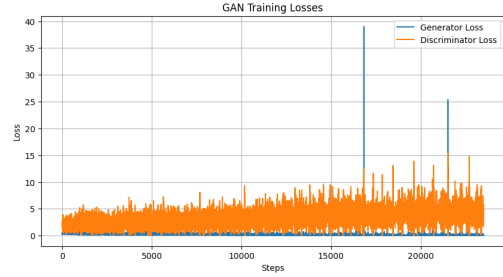


Fig. 11: DCGAN with Reduced Images Loss Function

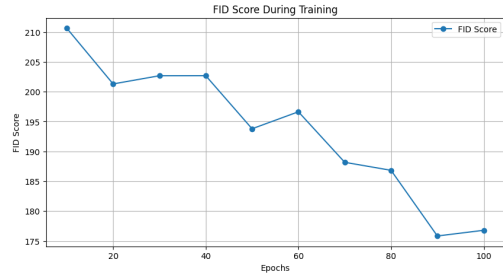


Fig. 12: DCGAN with Reduced Images FID

VII. DCGAN WITH MTCNN [1] FACE CROPPING

A. Image Preprocessing

To improve the quality of the images, we cropped the faces from our dataset by applying a Multi-Task Cascaded Convolutional Networks (MTCNN) [1] algorithm. The MTCNN uses a cascading series of neural networks to detect, align, and extract facial features from digital images with high accuracy and speed. We implemented MTCNN [1] using ‘opencv’ and ‘torchvision’, which allowed us to automatically crop the face regions from each image. After cropping, the resulting dataset was normalized and lightened to ensure consistency across it. This preprocessing step not only removed bloat from the background of the image but also enhanced the overall focus on facial features.

B. Generator Architecture

The proposed generator is an evolution of the previous model and is designed for a more complex GAN framework. It transforms a latent vector $z \in \mathbb{R}^{\text{latent_dim}}$ into a high-resolution 128×128 RGB image through two primary stages: a latent transformation and a sequence of convolutional blocks with upsampling.

1) Latent Transformation:

- **Input:** A latent vector $z \in \mathbb{R}^{\text{latent_dim}}$.
- **Linear Projection:** The latent vector is projected via a linear layer into a feature space of dimension $512 \times 4 \times 4$, where 4×4 represents the initial spatial resolution.

2) **Convolutional Blocks:** The projected feature map is processed through a series of convolutional blocks that gradually increase the spatial resolution while refining the feature representations. The detailed structure is as follows:

a) **Block 1:**

- **Preprocessing:** Batch Normalization is applied to the 512-channel feature map.
- **Upsampling:** The feature map is upsampled by a factor of 2 (from 4×4 to 8×8).
- **Convolution:** A Conv2d layer with 512 input and 512 output channels is applied using a 3×3 kernel, stride 1, and padding 1 (without bias).
- **Postprocessing:** The output is normalized with Batch Normalization, activated using LeakyReLU (negative slope 0.2), and regularized via Dropout2d with a dropout probability of 0.25.

b) **Block 2:**

- **Upsampling:** The feature map is upsampled from 8×8 to 16×16 .
- **Convolution:** A Conv2d layer transforms the feature map from 512 to 256 channels with the same kernel settings.
- **Postprocessing:** Batch Normalization, LeakyReLU activation (slope 0.2), and Dropout2d (dropout probability 0.25) are applied.

c) **Block 3:**

- **Upsampling:** The feature map is upsampled from 16×16 to 32×32 .
- **Convolution:** A Conv2d layer reduces the channel count from 256 to 128.
- **Postprocessing:** Batch Normalization, LeakyReLU activation (slope 0.2), and Dropout2d (dropout probability 0.2) are applied.

d) **Block 4:**

- **Upsampling:** The feature map is upsampled from 32×32 to 64×64 .
- **Convolution:** A Conv2d layer reduces the channels from 128 to 64.
- **Postprocessing:** The output is normalized using Batch Normalization, activated with LeakyReLU (slope 0.2), and regularized with Dropout2d (dropout probability 0.2).

e) **Block 5:**

- **Upsampling:** The feature map is upsampled from 64×64 to 128×128 .
- **Convolution:** A Conv2d layer further reduces the channel dimension from 64 to 32.
- **Postprocessing:** Batch Normalization and LeakyReLU activation (slope 0.2) are applied.

f) **Output Layer:**

- A final Conv2d layer maps the 32-channel feature map to 3 channels (RGB) using a 3×3 kernel with stride 1 and padding 1.
- A Tanh activation function is applied to ensure that the output pixel values lie within the range $[-1, 1]$.

Overall, this enhanced generator progressively improves and upsamples the initial latent representation. The use of Batch Normalization, LeakyReLU activations and Dropout regularization contributes to improved stability and performance in the complex GAN framework.

C. Discriminator Architecture

The improved discriminator incorporates channel attention and dual classification branches to enhance its discriminative power. It processes an input image through convolutional blocks, applies channel-wise attention, and then classifies the result using both a main and an auxiliary classifier.

a) **Convolutional Blocks:** The network begins with a series of convolutional blocks, each defined by a helper function. Each block consists of:

- **Convolution:** A Conv2d layer with a 4×4 kernel, stride 2, and padding 1.
- **Normalization:** Instance normalization (applied to all blocks except the first).
- **Activation:** A LeakyReLU activation with a negative slope of 0.2.

The sequential blocks are configured as follows:

- **Block 1:** Maps from 3 to 64 channels (without normalization).
- **Block 2:** Maps from 64 to 128 channels.
- **Block 3:** Maps from 128 to 256 channels.
- **Block 4:** Maps from 256 to 512 channels.
- **Block 5:** Maps from 512 to 512 channels.

b) **Channel Attention Module:** [15] The Channel Attention Module was incorporated into the discriminator to enhance feature discrimination by allowing the network to focus on the most informative channels. This is particularly beneficial in distinguishing subtle differences between real and generated images, thereby improving overall performance and training stability.

- Applies adaptive average pooling to squeeze the spatial dimensions of the feature map.
- Uses two 1×1 convolutions: the first reduces the channel dimensionality by a factor (default ratio 16) and is followed by a ReLU activation; the second restores the original channel size.
- A Sigmoid activation scales the channel weights, which are then multiplied by the feature map to emphasize informative features.

c) **Classification Layers:** The discriminator uses two parallel classifiers to produce a final validity score:

- **Main Classifier:**
 - Conv2d layer maps from 512 channels to 1 channel using a 4×4 kernel with stride 1 and no padding.
 - Sigmoid activation produces the primary validity score.
- **Auxiliary Classifier:**
 - The feature map is first flattened.
 - A Linear layer reduces the dimension from $512 \times 4 \times 4$ to 128.

- A LeakyReLU activation (negative slope 0.2) is applied.
- A subsequent Linear layer maps from 128 to 1, followed by a Sigmoid activation to yield an auxiliary validity score.

The final validity score is computed as the average of the outputs of the main classifier and the auxiliary classifier.

D. Loss Function

We maintained the same loss function from the previous GAN made.

E. Optimiser and Hyperparameters

The training uses the Adam optimizer for both the Generator and Discriminator with the following hyperparameters:

- Latent Dimension: 100
- Learning Rate: 0.00001
- Beta1: 0.5
- Beta2: 0.999
- Number of Epochs: 100

F. Training Outputs

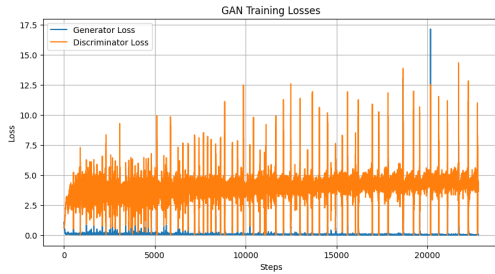


Fig. 13: DCGAN with MTCNN Face Cropping Training Losses

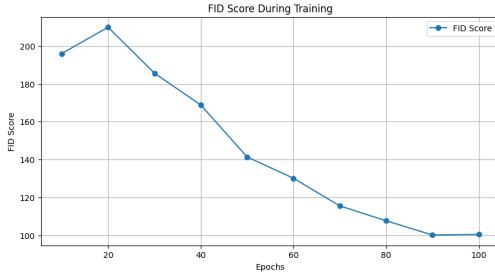


Fig. 14: DCGAN with MTCNN Face Cropping FID

VIII. DCGAN RESULT COMPARISON

In the training outputs of both DCGANs, we compared the losses of the Generator and Discriminator, alongside the visual quality of the generated images using the Fréchet Inception Distance (FID) [8]. FID compares the distributions of real and generated images by considering the image representations of the last pooling layer of the Inception v3 network.

Based on the graphs, several differences between the models are apparent. The simpler GAN exhibits more stability in the

generator loss, which could indicate better learning progress. However, the discriminator loss steadily increases and appears noisy, signaling some variability in the training process. In contrast, the more complex GAN shows a more stable and smoother progression for the discriminator after an initial increase, although it experiences occasional spikes. Meanwhile, the generator loss in this model worsens progressively, indicating that the generator is struggling to fool the discriminator.

When evaluating the models using FID, the distinction becomes even more evident. The simpler GAN produces a significantly higher FID [8] score compared to the more complex model. This suggests that the generated images from the second model are statistically closer to the real images' Gaussian distribution, indicating higher visual quality.

In conclusion, despite the simpler GAN showing better stability in training, the more complex GAN generates images that more closely resemble real data, as evidenced by its superior FID score.

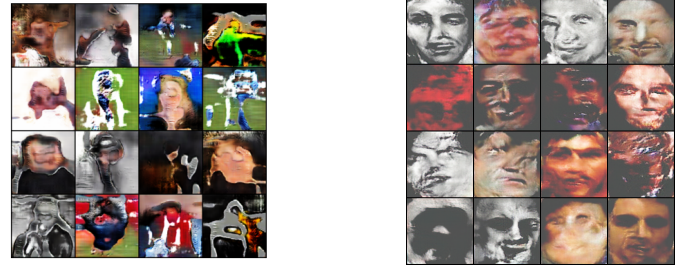


Fig. 15: DCGAN vs MTCNN + DCGAN

IX. VARIATIONAL AUTOENCODER (VAE)

A. Image Preprocessing

As referred previously, VAE will work with generated cropped images. Followed by the same preprocessing as the Simple DCGAN to possibly allow a faster and more stable learning process.

B. Encoder

The Encoder for the VAE model uses the cropped image (with 3 channels) and transforms them into a latent vector of size 128. Its architecture can be separated by four convolutional layers followed by two linear layers,

- **Convolution:** A Conv2d layer mapping from 3 to 32 channels using a 4x4 kernel size, stride of 2 and padding of 1.
- **Batch Normalization:** A BatchNorm2d function is applied
- **Activation:** A LeakyReLU activation function (negative slope of 0.2)
- **Convolution:** A Conv2d layer mapping from 32 to 64 channels using a 4x4 kernel size, stride of 2 and padding of 1.
- **Batch Normalization:** A BatchNorm2d function is applied

- **Activation:** A LeakyReLU activation function (negative slope of 0.2)
- **Convolution:** A Conv2d layer mapping from 64 to 128 channels using a 4x4 kernel size, stride of 2 and padding of 1.
- **Batch Normalization:** A BatchNorm2d function is applied
- **Activation:** A LeakyReLU activation function (negative slope of 0.2)
- **Convolution:** A Conv2d layer mapping from 128 to 256 channels using a 4x4 kernel size, stride of 2 and padding of 1.
- **Batch Normalization:** A BatchNorm2d function is applied
- **Activation:** A LeakyReLU activation function (negative slope of 0.2)
- **Flattening:** The output feature map (256,8,8) obtained from the convolutional layers is flattened into a one dimensional vector.
- **Linear:** Maps the vector input to a vector of size 256. This layer also produces the mean vector of the approximate posterior distribution. Each element of the mean vector represents the center of a dimension in the latent space.
- **Linear:** Similar to the previous layer, it maps the input vector to another vector of size 256. It also produces the logarithm of the variance for each dimension of the latent distribution. This log variance is used for numerical stability and a better gradient behavior.

C. Decoder

The Decoder for the VAE model takes a latent vector of size 128 and reconstructs the original image. Its architecture is separated into a linear transformation, a reshaping step and four convolutional layers:

- **Transposed Convolution:** A ConvTranspose2d layer mapping from 256 to 128 channels using a 4x4 kernel size, stride of 2 and a padding of 1.
- **Batch Normalization:** A BatchNorm2d is applied.
- **Activation:** A ReLU activation function is applied.
- **Transposed Convolution:** A ConvTranspose2d layer mapping from 128 to 64 channels using a 4x4 kernel size, stride of 2 and a padding of 1.
- **Batch Normalization:** A BatchNorm2d is applied.
- **Activation:** A ReLU activation function is applied.
- **Transposed Convolution:** A ConvTranspose2d layer mapping from 64 to 32 channels using a 4x4 kernel size, stride of 2 and a padding of 1.
- **Batch Normalization:** A BatchNorm2d is applied.
- **Activation:** A ReLU activation function is applied.
- **Transposed Convolution:** A ConvTranspose2d layer mapping from 32 to 3 channels using a 4x4 kernel size, stride of 2 and a padding of 1.
- **Activation:** A Sigmoid function is applied to limit the output values between 0 and 1.

D. Loss Function

The loss function used in the evaluation of the VAE is composed of two components: the binary Cross-Entropy loss [14] and the Kullback-Leibler Divergence (KLD) [16] term. The binary Cross-Entropy loss, which is also used in the evaluation of GANs, measures the reconstruction accuracy by comparing the generated output with the original input. The KLD [16] part acts as a regularizer, encouraging the learned latent distribution to be approximated to a standard normal distribution. This balance ensures that the model not only reconstructs the input data accurately but also maintains a smooth and well-structured latent space, which is essential for coherent sampling and interpolation.

$$D_{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Fig. 16: Kullback–Leibler divergence [16]

E. Optimiser and Hyperparameters

The training uses the Adam optimizer with the following hyperparameters:

- Latent Dimension: 100
- Learning Rate: 0.0001
- Beta1: 0.9
- Beta2: 0.999
- Number of Epochs: 100

F. Training Outputs

During the initial epochs, the loss begins at a high value, around 33250, but quickly decreases within the first 5 to 10 epochs. This decline clearly indicates that the VAE is rapidly learning to approximate the generated data to the real one, by reducing the reconstruction error and adjusting the Kullback–Leibler divergence term [16]. After this initial drop, this decline slows its pace down and around the 80th epoch stabilizes near 31250, suggesting that further epochs show marginal gains. Overall, the curve shows a consistent decrease in loss with no signs of overfitting, implying that the Variational Autoencoder is successfully learning to encode and decode the given data.

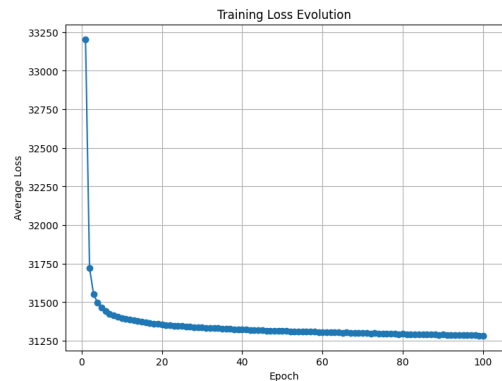


Fig. 17: VAE Training Loss

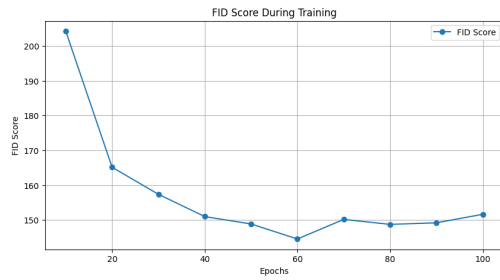


Fig. 18: VAE FID

X. MODEL COMPARASION: GAN VS VAE

To compare both models, we obviously compared the images generated and used the FID metric as the basis. From the previous FID graphs we observed throughout this report, we can conclude that the best absolute result we got was the combination of the DCGAN with the cropped images. This was what we expected since the beginning because the more complex image preprocessing focused solely on the face features we wanted to capture, the adversarial loss is more direct in punishing/rewarding the image generator, compared to the reconstruction loss from the encoder and the adversarial training makes the model capture high level details more effectively than the pixel level reconstruction made by the VAEs. These factors explain why the GAN, in the same image preprocessing conditions, provided better FID results by producing more realistic images, closer to the true data distribution.



Fig. 19: MTCNN + DCGAN vs MTCNN + VAE

XI. CONCLUSION

Throughout this project, we were able to learn and apply practical knowledge about image classification and generation by creating several different models based on convolutional neural networks, generative attention networks and variational autoencoders adapted to each objective. At the end of the classifier task, we conclude that CNNs are highly effective for image classification. Also, in some cases, image preprocessing can also enhance learning. Additionally, we found that overly complex models with too many parameters are prone to overfitting, making it difficult for them to generalize and accurately distinguish between classes. To address this, techniques such as dropout can be employed to reduce the risk of overfitting, or we can leverage the residual connections used in ResNet, which help mitigate this issue. One thing that we

were able to notice is that image processing is an important step to improve our results which can be seen especially for the generative task of the project. Overall, we believe that the objective of the project was achieved but we believe that, if we had better computational resources, better results could be achieved specially in the generative part of our project, where more iterations could cause the model to produce much better and more realistic images.

REFERENCES

- [1] The Modern Scientist, "Multi-Task Cascaded Convolutional Neural Network (MTCNN)", Medium. [Online]. Available: <https://medium.com/the-modern-scientist/multi-task-cascaded-convolutional-neural-network-mtcnn-a31d88f501c8>.
- [2] C. Ledig *et al.*, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network", arXiv preprint, 2017. [Online]. Available: <https://arxiv.org/abs/1609.04802>.
- [3] C. B. S., M. D. Shreyas, I. Karanth, B. H. S., A. Rani K P and G. S., "A Comparative Analysis on the Effectiveness of GAN Performance," 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), Delhi, India, 2023, pp. 1-8, doi: 10.1109/ICCCNT56998.2023.10307295. keywords: Training;Measurement;Visualization;Computational modeling;Semantics;Computer architecture;Generative adversarial networks;Machine Learning (ML);Generative Adversarial Networks (GANs);Image Generation;Discriminator;Generator;Comparative study,
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition", arXiv preprint, 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>.
- [5] Bishop, Christopher M., and Hugh Bishop. Deep Learning: Foundations and Concepts / Christopher M. Bishop, Hugh Bishop. Springer, 2024.
- [6] S. Scardapane, "Alice's Adventures in a Differentiable Wonderland – Volume I, A Tour of the Land", arXiv preprint, 2024. [Online]. Available: <https://arxiv.org/abs/2404.17625>.
- [7] H. Song, S. Huang, Y. Dong, and W.-W. Tu, "Robustness and Generalizability of Deepfake Detection: A Study with Diffusion Models", arXiv preprint, 2023. [Online]. Available: <https://arxiv.org/abs/2309.02218>.
- [8] TechTarget Contributor, "Fréchet Inception Distance (FID)", TechTarget. [Online]. Available: <https://www.techtarget.com/searchenterpriseai/definition/Frechet-inception-distance-FID>.
- [9] Asperti, Andrea. "Variance loss in variational autoencoders." International Conference on Machine Learning, Optimization, and Data Science. Cham: Springer International Publishing, 2020.
- [10] R. Kalantar, "Variational Auto-Encoder (VAE) - PyTorch Tutorial", Medium, 2021. [Online]. Available: <https://medium.com/@rekalantar/variational-auto-encoder-vae-pytorch-tutorial-dce2d2fe0f5f>.
- [11] University of Porto, "Moodle e-Learning Platform", 2024. [Online]. Available: <https://moodle2425.up.pt/>.
- [12] PyTorch, "ResNet — PyTorch Vision", PyTorch Hub. [Online]. Available: https://pytorch.org/hub/pytorch_vision_resnet/.
- [13] TensorFlow, "Classifier imagens com um DCGAN", TensorFlow Tutorials, 2024. [Online]. Available: <https://www.tensorflow.org/tutorials/generative/dcgan?hl=pt-br>.
- [14] PyTorch, "torch.nn.BCELoss", PyTorch Documentation, 2024. [Online]. Available: <https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html>.
- [15] Woo, Sanghyun, et al. "Cbam: Convolutional block attention module." Proceedings of the European conference on computer vision (ECCV). 2018.
- [16] A. Sharma, "Understanding KL Divergence", Towards Data Science, 2019. [Online]. Available: <https://towardsdatascience.com/understanding-kl-divergence-f3ddc8df254/>.